

Empirical Analysis of Transformers on Graph Connectivity

Part VII: Opening the Trunk — Why Does an Unbalanced Two-Chain Generalise?

Edoardo Paccagnella

July 9, 2026

Abstract

Report VI (Thread B) found a puzzle it could not explain: the same trained $L=2$ transformer, same weights, same canvas size $n=40$, solves a two-chain graph split into paths of length a and $n-a$ perfectly when a is small (e.g. $4+36$, correct out to the far end of the long path) but fails completely once the split is close to balanced (e.g. $17+23$, $20+20$ — correct only up to the architectural distance wall $3^L=9$, then exactly zero). Every report so far has read the model *behaviourally*: from its outputs, not from what it computes. This report opens the trunk — the attention weights, the read-out matrix, the per-layer embeddings — and asks what is mechanistically different between a split the model solves and one it does not.

1 Introduction

1.1 What Reports I–VI established

We treat the following as established (documented, multi-seed, in Reports I–VI); the new investigation must stay consistent with them.

- **There is a sharp, architectural distance wall at 3^L ($= 9$ at $L = 2$),** which moves with depth and appears even on graph families seen $\sim 10^8$ times in training (Reports III–IV).
- **The model computes distance-bounded matrix powering, not a bounded traversal:** a DFS/BFS reading is rejected on constructed and (partly) on natural graphs; a soft, learned “node budget” on dense structure is a removable data prior, not a second capacity (Report V).
- **Parallel routes and repeated local structure both help, within capacity, in ways that are data-shaped rather than architecturally free:** more disjoint routes between two endpoints rescue a connection the model could not otherwise confirm; a learned local rule (a bridge between two cliques) composes only weakly along a chain (Report VI).
- **The unexplained result this report targets:** sweeping the split $(a, n-a)$ of a two-chain graph at fixed $n = 40$, the same checkpoint goes from solving *every* pair in the long path (out to ~ 38 hops) to solving *none* beyond the 9-hop wall, as a crosses a threshold close to the capacity itself (Report VI, Thread B). No account of the mechanism was given.
- **Every report through Report VI is behavioural.** Model outputs are compared against oracles, ablated data distributions, or read-out variants — never against the attention weights or the read-out matrix themselves. `model.py` exposes tools for exactly that (real per-layer attention weights, per-layer embeddings) that no report has yet used.

1.2 The new question: what is mechanistically different?

Report VI’s own closing paragraph names this split puzzle as the one open thread it could reproduce but not explain. This report is the direct follow-up: instead of another behavioural sweep, we open the trained model at the level of its attention scores and its read-out weights, on the exact same checkpoints, and ask what changes between a split the model solves and one it does not.

1.3 Three candidate hypotheses

We state candidate explanations up front, so each experiment is a test that could come out either way — not a search for evidence of an assumed answer.

1. **Genuine propagation.** The model actually traces distance along the path; behaviour should be robust to relabelling nodes, to where the short component sits, and to the internal edge order of the long path.
2. **Short-component-resolved default.** The model fully resolves whichever component is small enough to fit inside its capacity (`short_len` $\lesssim 9$) and defaults every other pair to “connected” — a shortcut that is correct whenever exactly one component is small, and has nothing to fall back on once *both* components are too large to resolve.
3. **Label/order shortcut.** The model exploits some regularity in how node indices map to structure, rather than the abstract graph. (The existing evaluation already relabels every test graph independently at random before scoring it, which substantially weakens the crude form of this hypothesis before we even start; we still check it directly against the read-in/read-out weights.)

2 Setup

Model. The same base model as Reports III–VI: the RoBERTa-faithful standard transformer with the *linear* read-out, $L = 2$, single head, $d_{\text{model}} = 512$, normalised-ReLU attention $\alpha = (1/n) \text{ReLU}(QK^\top/\sqrt{d_h})$, trained online at $n = 40$ on a single clean distribution of disjoint paths (unions of 1–4 paths covering all n nodes), four seeds. This is the exact checkpoint set behind Report VI’s Thread B.

The forward pass, in full. Because §4.5–§4.7 open the trunk directly, we fix here, once, exactly what the model computes at inference, so that every quantity used later — the attention score, the normalised-ReLU weights, the message contribution, the node-to-node contribution — has a precise definition. The input is the self-loop-augmented adjacency $A' = A + I \in \{0, 1\}^{n \times n}$. The read-in, once,

$$H^{(0)} = \text{LayerNorm}(A' W_{\text{in}} + \mathbf{1} b_{\text{in}}^\top),$$

starts row i , before the LayerNorm, as simply the sum of one learned vector per neighbour of i (including i itself, via the self-loop) plus a bias — no attention yet. Each of the two transformer

blocks (layer $\ell = 1, 2$, post-LayerNorm) then maps $H^{(\ell-1)}$ to $H^{(\ell)}$ by

$$Q_\ell = H^{(\ell-1)}W_Q^{(\ell)}, \quad K_\ell = H^{(\ell-1)}W_K^{(\ell)}, \quad V_\ell = H^{(\ell-1)}W_V^{(\ell)} \quad (\text{single head, so } d_h = d_{\text{model}})$$

$$S_\ell = Q_\ell K_\ell^\top / \sqrt{d_h} \quad (\text{raw attention score})$$

$$\alpha_\ell = \frac{1}{n} \text{ReLU}(S_\ell) \quad (\text{normalised-ReLU attention: } \textit{not} \text{ a softmax, does not sum to 1})$$

$$\text{AttnOut}_\ell = (\alpha_\ell V_\ell) W_O^{(\ell)} \quad (\alpha_\ell \text{ weights the } \textit{values} V_\ell, \text{ then an output projection})$$

$$\hat{H}_\ell = \text{LayerNorm}(H^{(\ell-1)} + \text{AttnOut}_\ell) \quad (\text{residual})$$

$$H^{(\ell)} = \text{LayerNorm}(\hat{H}_\ell + \text{FFN}^{(\ell)}(\hat{H}_\ell)) \quad (\text{feed-forward, then residual})$$

Layer 2 repeats this starting from $H^{(1)}$, with its *own* W_Q, W_K, W_V, W_O and its own feed-forward weights — it does not reuse α_1 in any way; the two layers are two separate passes of this same recipe, run one after the other. The read-out (§2 below) is applied once, at the very end, to $H^{(2)}$.

Message contribution and node-to-node contribution: what they measure, and how they are computed. Two derived quantities recur in §4.5 and §4.7. The **message contribution** $\alpha_{ij}\|W_O v_j\|$ uses the actual value vector v_j and output projection W_O of one layer to size the additive message node j sends into node i ’s residual stream *at that layer* — it is exact for that one layer’s attention branch, but does not include the residual stream itself or the feed-forward step, both of which also update h_i . The **node-to-node contribution**

$$C_{ik} = \left\| \frac{\partial h_i^{(2)}}{\partial h_k^{(0)}} \right\|_F$$

answers the question directly, across *both* layers at once: how much does node i ’s final embedding move if node k ’s read-in embedding moves, holding everything else fixed? We compute it by literal automatic differentiation through the exact forward pass above — the real V, W_O , the real residual identity, the real feed-forward step, and LayerNorm’s true local derivative are all part of the computation, none of them dropped or approximated; the only sense in which this is “local” is the ordinary one for any derivative, that it is evaluated at one specific graph’s actual activations.

Why the read-out shape matters. With the linear read-out, the logit for pair (i, j) is $z_{ij} = h_i^\top w_j + b_j$, where w_j is the j -th row of the read-out matrix W_{out} — a fixed, learned vector attached to the *target* node j , not derived from j ’s own embedding. Node i ’s embedding h_i alone must therefore carry enough information to answer, for *every* other node j , “are we connected” — all the structure the model has learned about node j lives in w_j , fixed across every graph. This is why the read-out matrix itself, not only the attention weights, is a first-class object of study here.

Test graphs. Two-chain graphs partitioning all $n = 40$ nodes into two disjoint paths of size a and $n - a$, swept over the *full* range $a = 1, \dots, 20$ (Report VI reported only a handful of points). A new three-component construction — one small, resolvable path plus two large, individually-unresolvable paths that are *not* connected to each other — is used as a falsification probe for hypothesis 2 (§4.6).

3 Plan of experiments

Behavioural groundwork (§4.1–4.2). A dense split sweep and an explicit random-relabelling check, to fix the shape of the puzzle precisely and close off the crude label-shortcut

hypothesis before opening the trunk.

Read-out mechanism (§4.3–4.4). The $h_i^\top w_j$ decomposition and the geometry of W_{out} ’s rows — does the read-out encode component membership as a blob, or a graded function of distance?

Attention mechanism (§4.5). Real per-layer attention weights, the two-layer rollout, and the message contribution each node actually sends — does attention itself reach differently in a solved split than in a failed one?

Falsification test (§4.6). The three-component construction: if the model is really defaulting “not in the small component” to connected, it should wrongly merge two large components that are genuinely disconnected from each other.

4 Results

4.1 The split sweep, in full

Question. Report VI reported exact match at seven splits. Sweeping every $a = 1, \dots, 20$ locates the break precisely and lets us compare the model’s aggregate behaviour against a genuine distance-bounded oracle.

Result: the break sits one node above the capacity, and beyond it the model’s aggregate behaviour matches pure distance-bounded matrix powering almost exactly. Figure 1 (top) confirms Report VI’s shape at full resolution: exact match is ≈ 1.0 for $a \leq 7$ (with ordinary sampling noise at $a = 3, 6$: 0.80–0.87, consistent across all four seeds, not a single bad run), collapses to exactly 0 from $a = 8$ on, and the long-component reach follows the same break with a soft floor-plus-recovery from $a \approx 11$ to $a = 20$ — the familiar shape of every beyond-capacity curve in this project. The new observation is the dashed and dotted curves: the model’s overall predicted-positive rate (fraction of all pairs called “connected”) tracks the *true* positive rate closely while $a \leq 7$, then drops sharply and, from $a \approx 11$ on, sits almost exactly on top of the positive rate a pure distance- ≤ 9 oracle would produce on the same graphs (0.36 measured vs. 0.35 from the oracle, flat in a because a long path’s 9-hop neighbourhood size does not depend on how far away the path’s other end is). **Once both components are individually beyond capacity, the model’s aggregate behaviour is indistinguishable from plain distance-bounded matrix powering;** the extra “complete the whole long component” behaviour is confined to the regime where one side is small.

split ($a, 40-a$)	exact	reach (long)	reach (short)	cut	pred. positive rate
(1, 39)	1.000	1.000	—	1.000	0.951
(4, 36)	1.000	1.000	1.000	1.000	0.820
(7, 33)	0.779	1.000	1.000	0.998	0.712
(8, 32)	0.000	0.992	1.000	0.925	0.699
(9, 31)	0.000	0.873	1.000	0.938	0.599
(10, 30)	0.000	0.648	1.000	0.999	0.434
(14, 26)	0.000	0.581	0.889	1.000	0.362
(17, 23)	0.000	0.638	0.792	1.000	0.362
(20, 20)	0.000	0.709	0.709	1.000	0.362

Table 1: *Model*: base standard transformer with the linear read-out ($L = 2$, single head, $d_{\text{model}} = 512$), trained on a single clean distribution of disjoint paths, four seeds, $n = 40$ (the checkpoints behind Report VI’s Thread B). *Test set*: two-chain graphs split $(a, 40-a)$, 200 freshly sampled, independently node-relabelled graphs per split per seed (800 total per split). *Metrics* (mean over seeds): whole-graph exact match; pairwise reach inside the long/short path (target connected; “—” at $a = 1$, no internal short-path pairs); pairwise accuracy across the two paths (target disconnected); the model’s overall fraction of pairs predicted connected. Full sweep $a = 1, \dots, 20$ in Figure 1.

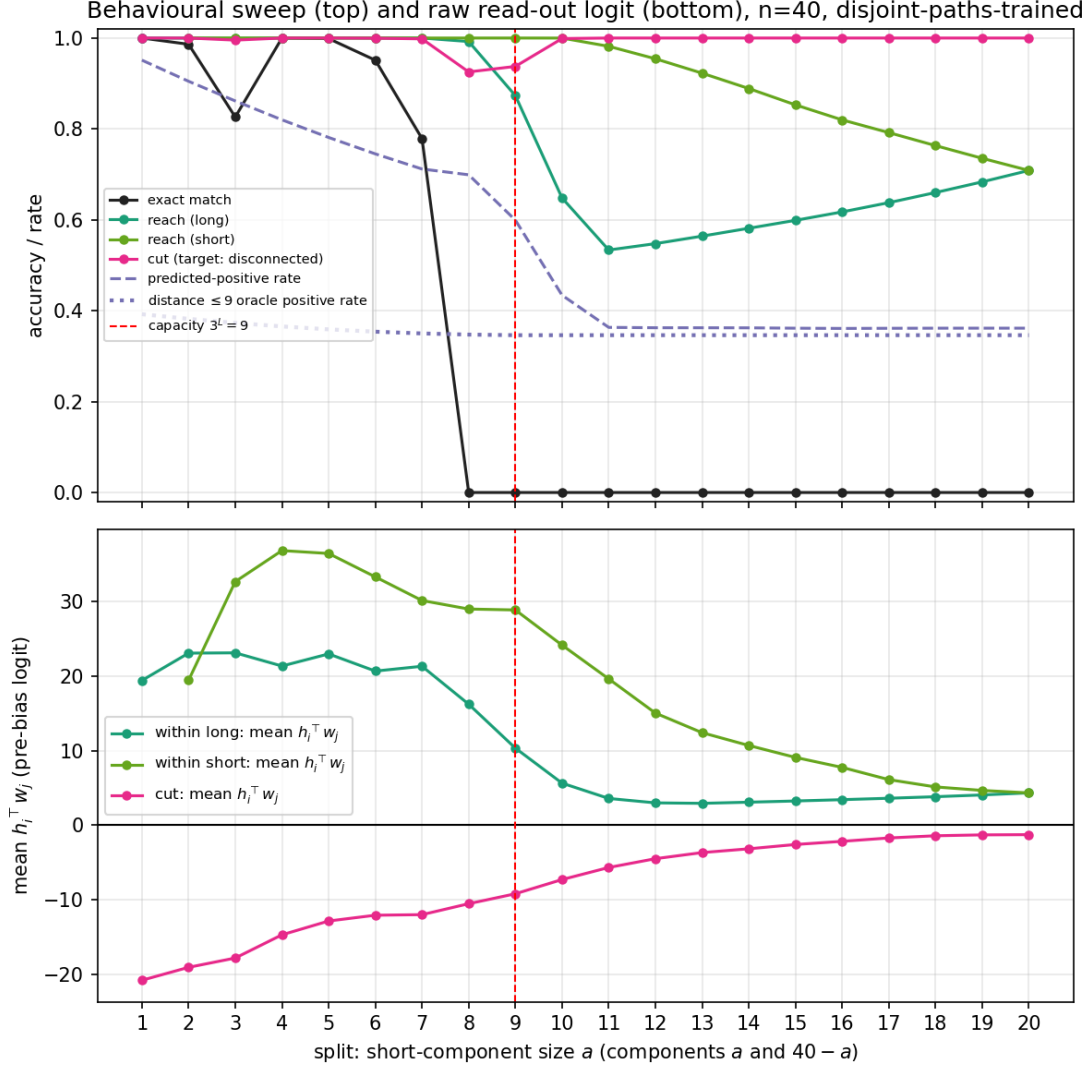


Figure 1: *Top: Model/training/test as in Table 1, full split sweep $a = 1, \dots, 20$; the dashed line is the model’s overall predicted- positive rate, the dotted line the positive rate a pure distance- ≤ 9 oracle would give on the identical graphs (both computed in closed form from the two path lengths, independent of any model). Bottom: same model and test graphs, the mean raw read-out logit $h_i^\top w_j$ (before adding the target’s bias b_j ; §2) for pairs within the long path, within the short path, and across the two paths, mean over seeds. Red dashed line: capacity $3^L = 9$.*

The collapse is not a discontinuity in the mechanism. Figure 1 (bottom) is the reason the exact-match curve looks like a cliff: the raw, pre-bias logit $h_i^\top w_j$ for a pair inside the long path is not a switch, it is a smoothly decaying quantity. It sits saturated and strongly positive for $a \leq 7$ (≈ 17 – 36), falls through zero across $a = 8$ – 11 (exactly where accuracy collapses), and settles into a small, slowly *recovering* positive value from $a \approx 11$ to $a = 20$ — the same soft floor-plus-recovery shape as the accuracy curve above it, now visible in the model’s raw confidence rather than only in its thresholded predictions. The cut logit (across the two paths, target disconnected) tells a different story: it starts very negative (≈ -21 at $a = 1$) and its magnitude shrinks steadily as the split balances, but it *never crosses zero* at any split tested — the “different component” decision degrades in confidence but not in sign, consistent with cut accuracy staying at ≈ 1.0 across the whole sweep in Table 1.

4.2 Relabelling does not explain it

Every graph in §4.1 is already independently relabelled at random before being scored (a fresh random permutation per graph, unpermuted before comparing predictions) — this is the existing methodology, not a new control. We add one direct comparison: the same split sweep with node labels *fixed* to a canonical order (the short path always occupying the first a indices) instead of randomised. Table 2 shows no meaningful difference at any split. Hypothesis 3 in its crude form — the model exploiting a fixed mapping from node index to structure — is not what is happening here.

split ($a, 40-a$)	exact, random relabelling	exact, fixed labelling
(1, 39)	1.000	1.000
(4, 36)	1.000	1.000
(7, 33)	0.779	0.750
(8, 32)	0.000	0.000
(10, 30)	0.000	0.000
(17, 23)	0.000	0.000
(20, 20)	0.000	0.000

Table 2: *Model/training as in Table 1. Test set:* two-chain graphs at the split shown, 200 graphs per seed, either independently relabelled at random per graph (the condition behind every other result in this report) or with node labels fixed to a canonical order (short path first). *Metric:* whole-graph exact match, mean over four seeds.

4.3 What the read-out logit actually responds to

§4.1 already showed the headline pattern (Figure 1, bottom): the within-long logit is large and saturated while the short component is resolvable, collapses through the transition, and settles at a small, recovering value that mirrors distance-bounded matrix powering. The within-short logit is the mirror image by construction (symmetric around $a = 20$, where the two logits meet exactly). One additional check confirms the decomposition is measuring the right quantity: aggregating $h_i^\top w_j + b_j > 0$ by pair type reproduces the behavioural reach numbers of Table 1 to within sampling noise at every split (e.g. 0.650 vs. 0.648 at $a = 10$, 0.709 vs. 0.709 at $a = 20$) — the two independently-computed views of the same model agree exactly, as they must.

4.4 The geometry of the read-out matrix

If the model relied on a fixed node-index shortcut (hypothesis 3), we would expect a target-specific classifier vector w_j to look unusually different from its neighbours, or the direct read-in-to-read-out skip connection $E_{\text{in}}W_{\text{out}}^\top$ (bypassing both transformer layers entirely) to already carry most of the signal. Neither holds (Table 3): the w_j row norms are uniform across all 40 targets (standard deviation $\approx 4\%$ of the mean, every seed), the pairwise cosine similarity between different w_j ’s is small (≈ 0.06), and the skip-connection alignment is small everywhere (diagonal ≈ 0.2 – 0.3 , off-diagonal ≈ 0) — two orders of magnitude below the $h_i^\top w_j$ values of 10–35 that actually decide a prediction (§4.3). Whatever produces the large, split-dependent logit swings is built by the two attention/feed-forward layers, not read directly off which node is whose neighbour.

	$\ w_j\ $ (mean, std)	mean $\cos(w_j, w_k)$ ($j \neq k$)	$E_{\text{in}}W_{\text{out}}^\top$	
			diagonal	off-diagonal
seed 1000	5.18 ± 0.23	+0.065	+0.31	-0.03 ± 0.17
seed 2000	5.21 ± 0.19	+0.063	+0.25	-0.06 ± 0.19
seed 3000	5.09 ± 0.19	+0.063	+0.26	-0.02 ± 0.17
seed 4000	5.12 ± 0.20	+0.061	+0.24	$+0.03 \pm 0.16$

Table 3: *Model*: the four disjoint-paths-trained checkpoints of Table 1 (no test graphs — these are static properties of the trained weights). *Metrics*: the read-out row norms $\|w_j\|$ (mean $\pm$ std. over the 40 target nodes); the mean pairwise cosine similarity between different rows of W_{out} ; and the read-in/read-out skip-connection alignment $E_{\text{in}}W_{\text{out}}^\top$ (row k of the read-in weight against row j of the read-out weight), split into its diagonal ($k = j$) and off-diagonal mean $\pm$ std.

4.5 Attention: where the model actually looks

Question. The two-layer attention rollout $M = \alpha_2\alpha_1$ approximates, for each query node, how much of its final representation traces back to each other node. Does this quantity reach differently in a split the model solves than in one it does not?

Result: attention reaches every node of the long component either way; what changes is how well it stays inside it. For a node roughly in the middle of the long path, the rollout already touches every other node of the long component at $a = 4$ and at $a = 20$ (all 36, respectively all 20, long-component nodes receive non-negligible rollout mass) — attention is not simply “too short-range” to reach far nodes in the failing case. What differs is how much of that same node’s rollout mass leaks onto the wrong (short) component: at $a = 4$ essentially none of it does (2.9% of the long-vs-short mass, illustrative single-node example); at $a = 20$ nearly a third does (29%). Figure 2 shows this is not a one-node anecdote: pooled over every long-component node, the leak fraction rises smoothly and monotonically from 0.3% at $a = 1$ to 31% at $a = 20$, tight across all four seeds at every split (spread < 1 percentage point), tracking the same transition zone around the capacity as every other quantity in this report. **The attention mechanism does not fail to reach far nodes as the split balances — it fails to keep its reach confined to the query’s own component.**

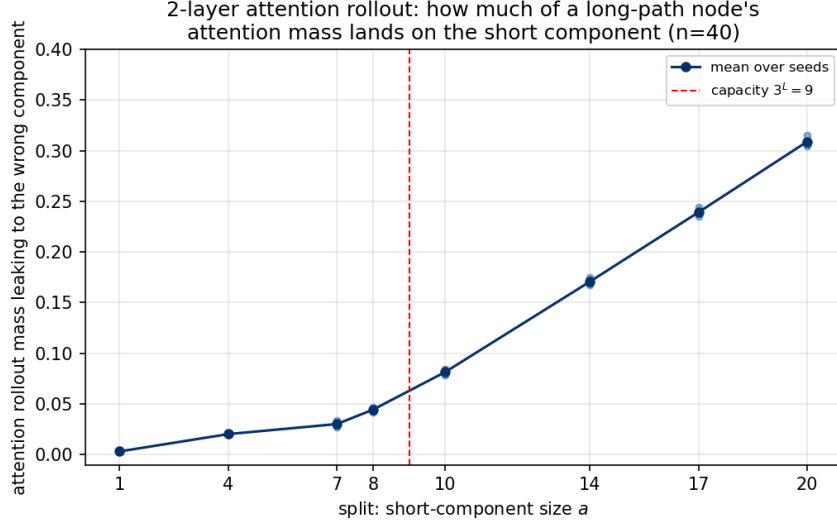


Figure 2: *Model/training as in Table 1. Test set:* two-chain graphs at the splits shown, 60 graphs per seed. *Metric:* for each long-component node, the fraction of its two-layer attention-rollout mass ($\alpha_2\alpha_1$, real attention weights via the model’s attention maps) that lands on the short component rather than its own long component, mean over long-component nodes; small dots are individual seeds, the line is the mean over seeds. Red dashed line: capacity $3^L = 9$.

4.6 Falsification test: does the model actually merge two unrelated large components?

Question. If the model’s mechanism in §4.1–4.5 were simply “resolve whichever component is small, default everything else to connected” (hypothesis 2 in its crude form), then on a graph with *three* disjoint paths — one small and two large, the two large ones genuinely disconnected from each other — the model should wrongly call the two large components connected to each other (both read only as “not the small one”).

Result: the crude shortcut is refuted, but so is a full recovery — the completion trick is specific to a two-component canvas. It is not refuted by a coin flip: at every small-component size tested and in every one of the four seeds, the model correctly calls the two large components disconnected (cut = 1.000 throughout, Table 4) — the “which component” decision generalises cleanly to three components, not just two. But the within-large-component reach does *not* recover to the near-perfect values §4.1 found for a single long path beside a small stub: it sits at 0.71–0.86, matching almost exactly the degraded reach of a *balanced two-way* split of a similar size (Table 1: 0.71 at the balanced (20,20) split). Adding a small, cleanly-resolvable third component does not rescue the two large components’ internal connectivity — each is treated as if it were the “long” side of a balanced two-way split, not as the sole *other* component in the graph. The completion trick found in §4.1 is specific to graphs with exactly two components, one of them small; it is not a general “not-small therefore connected” rule.

small	large ₁	large ₂	reach large ₁	reach large ₂	cut(S,L ₁)	cut(S,L ₂)	cut(L ₁ ,L ₂)
1	19	20	0.737	0.710	1.000	1.000	1.000
2	19	19	0.737	0.737	1.000	1.000	1.000
4	18	18	0.765	0.765	1.000	1.000	1.000
7	16	17	0.825	0.794	1.000	1.000	1.000
8	16	16	0.824	0.825	1.000	1.000	1.000
10	15	15	0.855	0.855	1.000	1.000	1.000

Table 4: *Model/training as in Table 1. Test set:* three disjoint paths partitioning all 40 nodes — one small path (size shown) and two large paths (sizes large₁, large₂), the two large paths *not* connected to each other; 300 graphs per seed, independently node-relabelled. *Metrics* (mean over four seeds, tight in every cell): pairwise reach inside each large path (target connected); pairwise accuracy between the small path and large₁/large₂, and between the two large paths (all three target disconnected — “cut(L₁,L₂)” is the decisive column).

4.7 Attention and read-out weights, visualised

The attention pattern itself changes shape between a solved and a failed split. Figure 3 shows the real attention scores $S = QK^\top/\sqrt{d_h}$ and the resulting normalized-ReLU α , per layer, for one example graph at $a = 4$ and one at $a = 20$. Layer 0 is essentially identical in both: a narrow band along the diagonal (each node attending to its immediate path neighbours), broken cleanly at the component boundary — a purely local, structure-driven pattern that does not depend on the split. Layer 1 is where the two conditions diverge. At $a = 4$, the rollout $\alpha_2\alpha_1$ (Figure 4) is a single, near-uniform bright block covering the *entire* long component — every node’s representation draws on every other long-component node. At $a = 20$, the same rollout is visibly block-diagonal: two separate, moderately-bright squares (one per half) with a darker, attenuated region between them — exactly the attention-level picture behind the leak-fraction curve of Figure 2. The row-mass panels (right column, Figure 4) make the shape concrete: at $a = 4$, layer-1 row-mass rises once past the component boundary and stays on a flat, high plateau for the whole long path; at $a = 20$, it rises after *each* boundary but dips again in the middle of each half, a double-hump rather than a plateau. Per-node $Q/K/V$ vector norms (not shown in the main figures; computed for every split) are essentially *identical* in shape between $a = 4$ and $a = 20$ — the difference is not in how strongly a node’s query/key/value is scaled, only in which other nodes’ keys and values it lines up with.

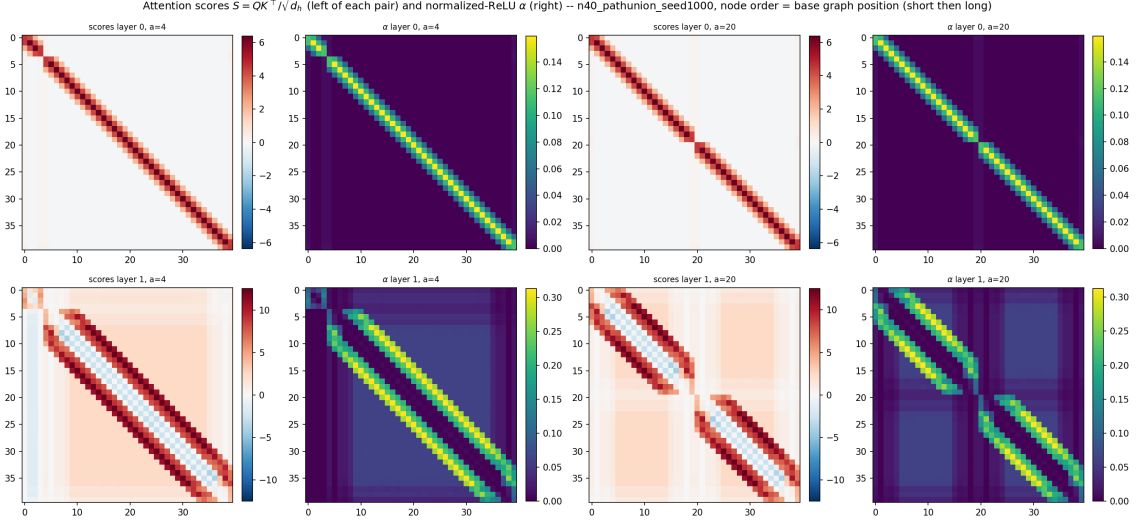


Figure 3: *Model*: one seed of the checkpoints of Table 1 (base standard transformer, linear read-out, $L = 2$, single head, $d_{\text{model}} = 512$, disjoint-paths-trained, $n = 40$). *Test set*: one representative two-chain graph at $a = 4$ and one at $a = 20$, node order fixed to base graph position (short path first, then long) for readability. *Metric*: the real attention score matrix $S = QK^\top / \sqrt{d_h}$ and the resulting normalized-ReLU attention $\alpha = \text{ReLU}(S)/n$, layer 0 (top) and layer 1 (bottom), averaged over 80 independent random relabellings of the same underlying graph then mapped back to base node order (so the figure reflects the model’s stable behaviour on this graph shape, not one noisy instance).

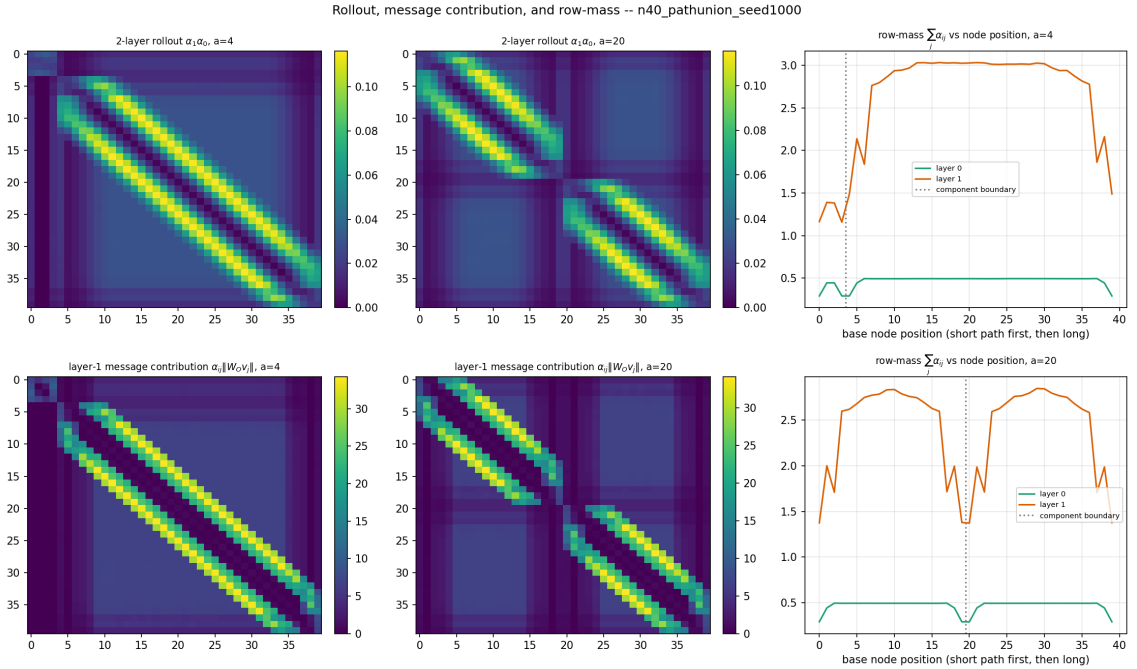


Figure 4: *Model/test as in Figure 3*. *Metric*: the two-layer attention rollout $\alpha_2\alpha_1$ (top row) and the layer-1 message contribution $\alpha_{ij}\|W_{ov_j}\|$ (bottom row) — how much node j ’s value, after the output projection, actually adds to node i ’s residual stream — for $a = 4$ (left) and $a = 20$ (middle); and the row-mass $\sum_j \alpha_{ij}$ (right column) along the chain, layer 0 vs. layer 1, for both splits, dotted line marks the component boundary.

The read-out and read-in weight matrices show no exploitable structure (further evidence against hypothesis 3). Figure 5 renders what Table 3 already summarised in numbers: W_{out} and W_{in} look like unstructured noise at the level of individual entries, the cosine-similarity matrices between different target/label vectors are uniformly small off the diagonal, and the skip-connection alignment $E_{\text{in}}W_{\text{out}}^{\top}$ has no diagonal or banded pattern beyond a few scattered entries — there is no visible “special” node index, and no sign that the model precomputes an index-based shortcut anywhere in these two matrices. The singular-value spectra of every weight matrix in the trunk (Figure 6) decay smoothly with no isolated dominant direction, consistent with a distributed computation rather than a single low-rank shortcut circuit.

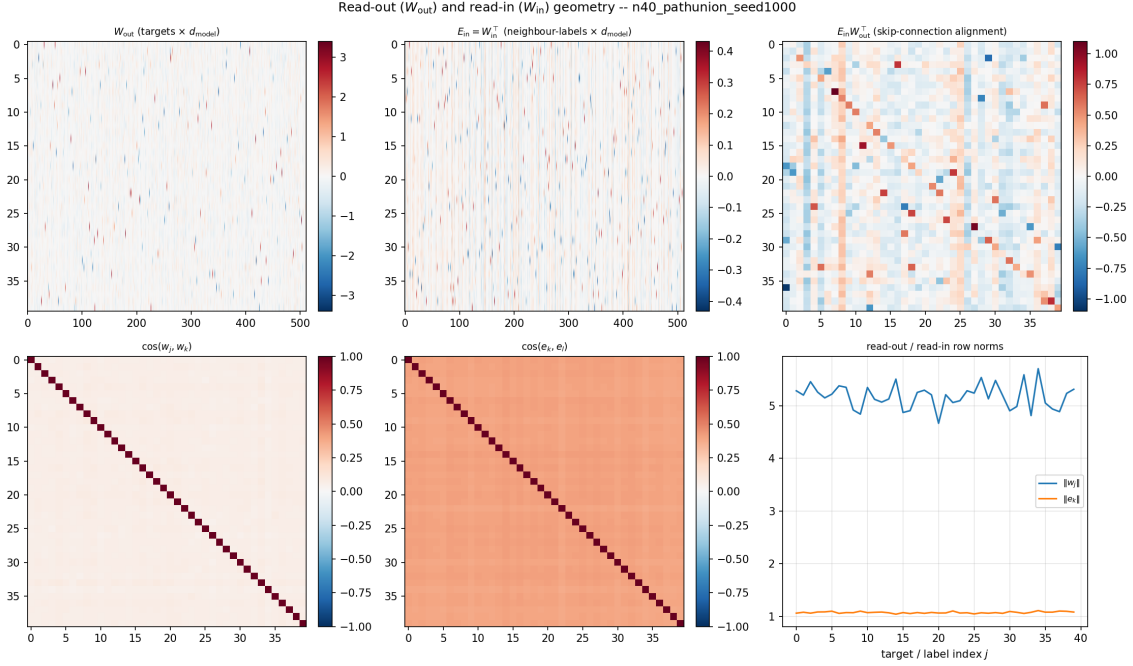


Figure 5: *Model*: the four checkpoints of Table 3 (static weights, no test graphs). *Metric*: W_{out} and $E_{\text{in}} = W_{\text{in}}^{\top}$ (raw entries); the skip-connection alignment $E_{\text{in}}W_{\text{out}}^{\top}$; the cosine-similarity matrices $\cos(w_j, w_k)$ and $\cos(e_k, e_l)$; and the row norms $\|w_j\|, \|e_k\|$ by target/label index (one representative seed shown).

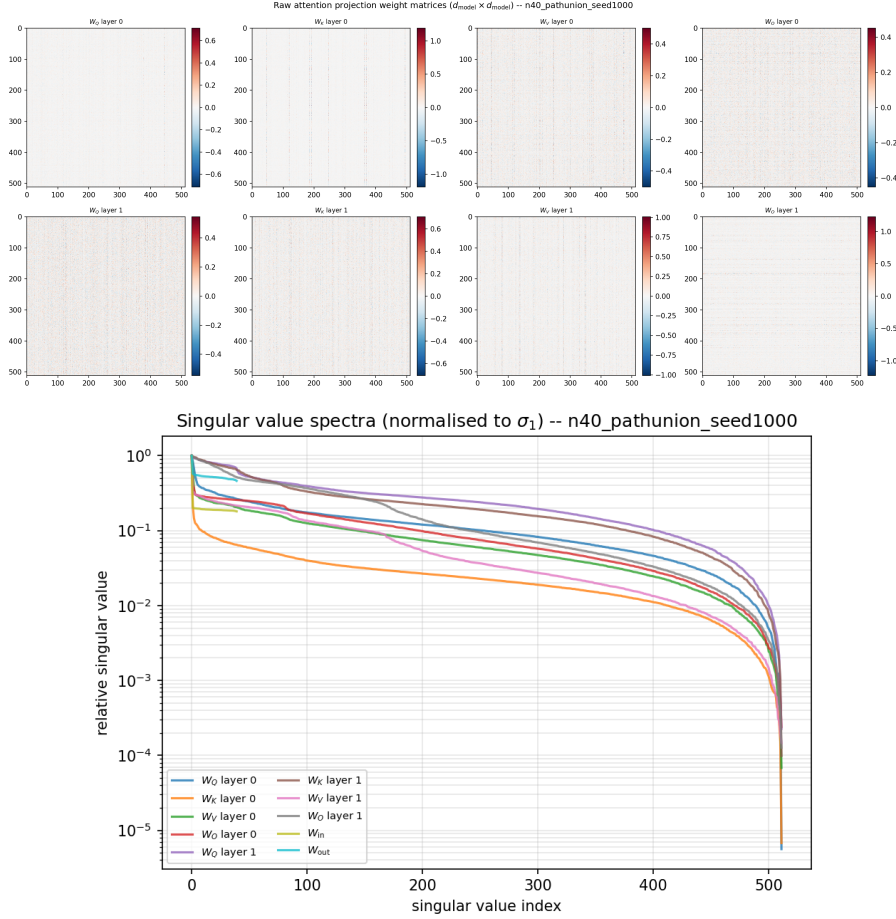


Figure 6: *Model as in Figure 5. Top: the raw attention projection matrices W_Q, W_K, W_V, W_O for both layers ($d_{\text{model}} \times d_{\text{model}}$). Bottom: singular value spectra of every trunk weight matrix, normalised to the largest singular value (one representative seed).*

4.8 Localising the completion signal: layer ablations

Question. Which part of the trunk is responsible for the extra, split-specific completion signal — and is it the same machinery that does ordinary distance-bounded reach, or a separate add-on?

Setup. Starting from an otherwise normal forward pass, we zero one branch of the residual stream at a time — the attention output or the feed-forward output, at layer 0 or layer 1 — and re-run the full split sweep; a fifth condition bypasses both transformer blocks entirely (read-out applied directly to the read-in embedding, no cross-node mixing at all). If ablating a component collapses the $a \leq 7$ behaviour toward the ablated baseline everywhere, that component is doing the work; if the ablated model is essentially unaffected, it is not.

Result: reach and cut depend on different parts of the trunk, and the completion signal is not a separate circuit — it needs the same attention layers as ordinary reach. Figure 7 and Table 5 show a clean split. **Either** attention layer alone is necessary for reach: zeroing layer 0’s attention collapses reach to 0.06–0.11 at *every* split, including $a \leq 7$; zeroing layer 1’s collapses it to 0.15–0.29 — both far below even the ordinary beyond-capacity floor, and both close to the no-mixing-at-all bypass baseline (0.32–0.34, flat in a). The $a \leq 7$ completion signal does not survive losing either attention layer any better than the ordinary $a \geq 11$ regime does: it is built by the *same* two-layer attention machinery, not a separate

mechanism riding on top of it. The feed-forward branches tell a different, sharper story. Zeroing layer 1’s feed-forward leaves reach almost untouched (0.87–0.99, barely below baseline at every split) but collapses *cut* from ≈ 1.0 to 0.26–0.62 once the split is no longer tiny — **cut and reach are read out through different final-layer computations**: cut depends heavily on layer 1’s feed-forward step, reach barely does. Layer 0’s feed-forward instead matters more for reach in the completion regime specifically (baseline 1.0 $\rightarrow$ 0.47 at $a = 7$ under ablation, a much larger relative hit than the 0.71 $\rightarrow$ 0.59 it takes at $a = 20$) while cut is only mildly affected throughout.

condition	$a=1$	$a=4$	$a=7$	$a=8$	$a=10$	$a=14$	$a=17$	$a=20$
<i>reach (long component)</i>								
baseline	1.000	1.000	1.000	0.993	0.651	0.581	0.638	0.709
zero attn 0	0.058	0.063	0.068	0.070	0.074	0.084	0.094	0.107
zero attn 1	0.154	0.168	0.182	0.187	0.199	0.226	0.253	0.288
zero mlp 0	0.808	0.635	0.469	0.428	0.438	0.490	0.535	0.593
zero mlp 1	0.992	0.934	0.925	0.916	0.912	0.899	0.885	0.869
bypass	0.322	0.325	0.326	0.328	0.326	0.332	0.332	0.336
<i>cut (target: disconnected)</i>								
baseline	1.000	1.000	0.998	0.926	0.999	1.000	1.000	1.000
zero attn 0	0.995	0.995	0.994	0.994	0.993	0.993	0.993	0.993
zero attn 1	0.986	0.997	0.996	0.996	0.995	0.995	0.993	0.994
zero mlp 0	0.969	0.887	0.898	0.945	0.942	0.917	0.914	0.912
zero mlp 1	0.939	0.599	0.617	0.618	0.534	0.276	0.262	0.256
bypass	0.660	0.692	0.689	0.693	0.688	0.691	0.692	0.691

Table 5: *Model/training as in Table 1*, four seeds. *Test set*: two-chain graphs at the splits shown, 150 graphs per seed, independently node-relabelled. *Metric*: reach (long component, top block) and cut (bottom block), mean over seeds, under six forward-pass conditions — unmodified (baseline); the attention or feed-forward branch zeroed at layer 0 or 1; and both transformer blocks skipped entirely (bypass, read-out applied directly to the read-in embedding).

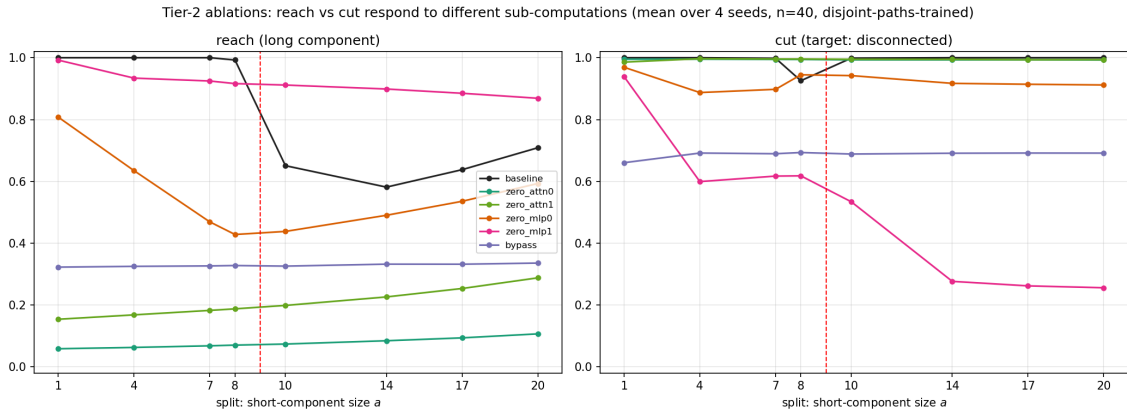


Figure 7: Table 5 as curves. *Model/training/test as in Table 5*, mean over four seeds. Red dashed line: capacity $3^L = 9$.

4.9 Activation patching between adjacent splits (exploratory)

Question. Can transplanting part of a *solved* forward pass into a *failing* one rescue the failing prediction — and if so, does it matter *which* nodes are transplanted?

Setup. Two different splits are two different graphs, so there is no generic node-for-node correspondence between e.g. $a = 4$ and $a = 20$. We align nodes by *hops from the component boundary* instead of raw index — the one correspondence that is meaningful across two split sizes — under fixed (canonical, unpermuted) node labelling for exact alignment. We patch a solved *donor* graph’s residual stream into a failing *recipient* graph’s forward pass, at the boundary-aligned node positions only, at two points (right after block 0, and at the final embedding before the read-out), and compare against a control that patches the same *number* of nodes but at *random*, non-aligned positions.

Result: patching works, but not because of boundary alignment — the read-out’s per-row structure does most of the work, and only the earlier patch point is sensitive to *which* donor is used. Table 6 shows patching the *final* embedding rescues reach to 1.0 in every case tested — but so does the random-position control, almost as well (0.89–1.0). This is a direct, expected consequence of the read-out architecture (§2): since $z_{ij} = h_i^\top w_j + b_j$ depends only on row i , copying a solved graph’s h_i into most rows reproduces solved-*like* predictions for those rows almost by construction, whether or not the copied rows are the “right” ones. The final-embedding patch is therefore not good evidence for boundary alignment specifically. The earlier patch point (right after block 0, before block 1 reprocesses it) is more informative precisely because it is *not* always helpful: for the adjacent, structurally similar pair ($7 \rightarrow 10$) it rescues reach to 0.98, close to the aligned donor; for the mismatched pair ($4 \rightarrow 20$) — where the donor and recipient have very different short/long proportions — patching at this earlier point *hurts*, dropping reach to 0.31, well *below* the recipient’s own unpatched baseline (0.71). Block 1’s attention, reprocessing a hybrid state built from two differently-shaped graphs, is confused rather than helped. Taken together: this experiment does not cleanly localise the completion signal to the boundary region — the final-layer result is largely an artefact of the read-out’s per-row structure, and the one result that *is* alignment-sensitive (the block-0 patch) shows fragility to a topology mismatch rather than a clean rescue. We report it as exploratory and inconclusive rather than decisive, and do not read more into it than that.

donor $\rightarrow$ recipient	aligned nodes	baseline (recipient)	patch (final embedding)	patch (after block 0)	random control
$a=7 \rightarrow a=8$	39	0.989	1.000	1.000	1.000
$a=7 \rightarrow a=10$	37	0.626	1.000	0.976	0.976
$a=4 \rightarrow a=20$	24	0.709	1.000	0.307	0.889

Table 6: *Model/training as in Table 1*, four seeds, one canonical (fixed-label) graph pair per seed per row (deterministic, not sampled). *Metric:* reach (long component) of the recipient split, mean over seeds, before patching (baseline) and after patching the donor’s residual stream into the boundary-aligned recipient nodes at two points, vs. a same-count random-position control.

4.10 Generalising to $n = 64$: path_union and ER

Question. Every result so far is on one canvas size, $n = 40$. Does the same completion signal appear at a larger, sparser canvas, and does it depend on having seen disjoint-path structure in training at all? We repeat the full Tier 1/2 audit on the $n = 64$ checkpoints from Report VI’s Thread A — already trained, no new training needed — for two training distributions: the same disjoint-paths family as everywhere else in this report, and pure sparse Erdős–Rényi (which never contains a multi-path or two-chain graph).

Result: ER shows no completion signal at all, and tracks the distance oracle almost exactly at every split. Figure 8 is the cleanest single result in this report. Trained purely

on ER, the model shows *no* special behaviour at small a : reach climbs smoothly from 0.21 to 0.39 as a goes from 1 to 32, sitting within a few points of the distance- ≤ 9 oracle rate at *every* split, with no cliff, no floor, no recovery — exactly the profile of plain distance-bounded matrix powering, never departing from it. Cut, meanwhile, stays at ≥ 0.997 across the entire sweep, including the fully balanced split. The completion signal found throughout this report is not an architectural inevitability of the trunk — it is something the disjoint-paths training distribution specifically teaches; a model that never saw that structure does not acquire it, at any split.

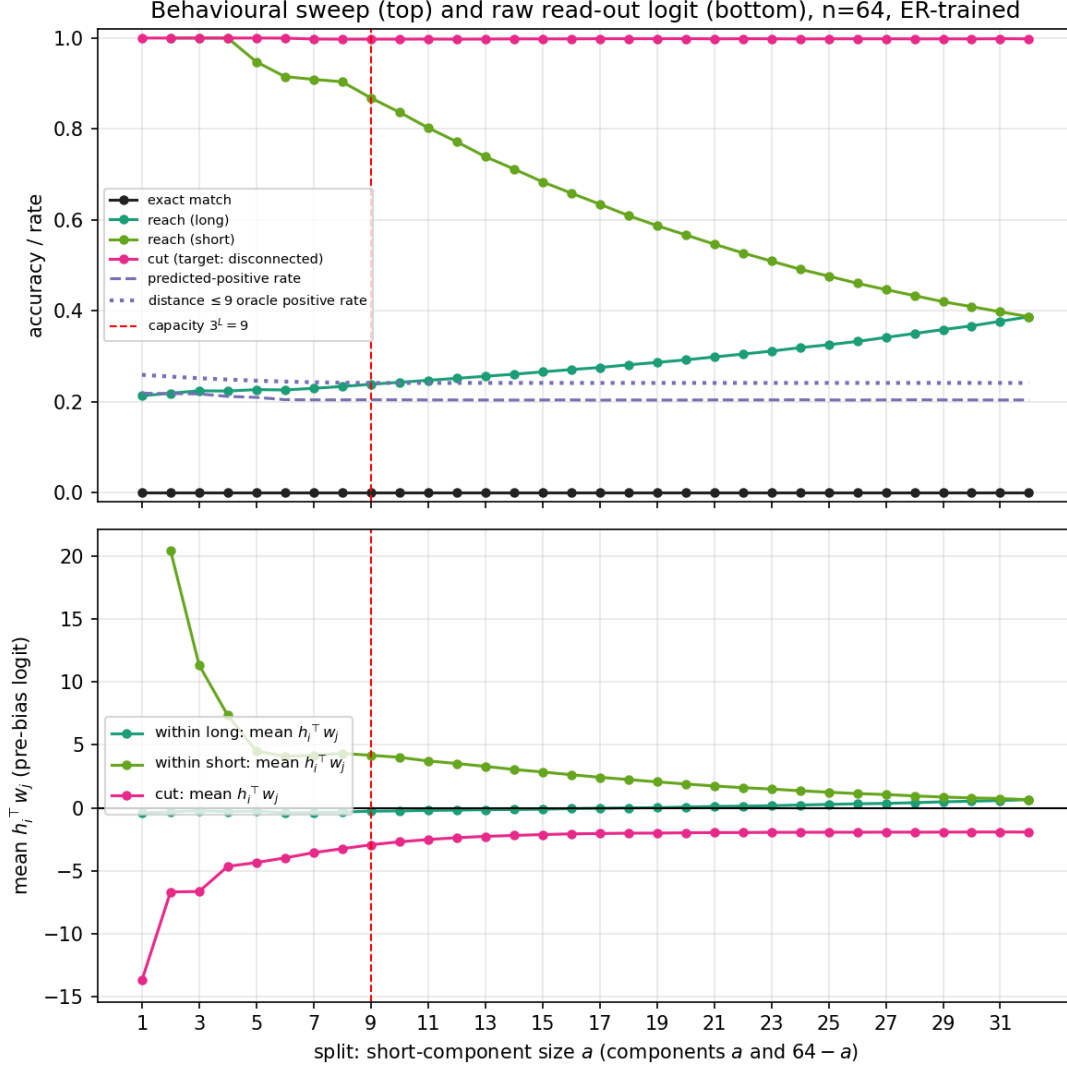


Figure 8: *Model*: base standard transformer, linear read-out ($L = 2$, single head, $d_{\text{model}} = 512$), trained on pure sparse Erdős-Rényi ($p = 0.03$, mean degree ≈ 1.9 ; never any path structure), four seeds, $n = 64$ (checkpoints from Report VI Thread A). *Test set/metrics as in Figure 1*, full split sweep $a = 1, \dots, 32$.

Result: path_union at $n = 64$ reproduces the completion signal for reach, but the failure mode beyond it inverts — cut collapses instead of staying robust. Figure 9 shows the disjoint-paths-trained $n = 64$ model does still single out small a (exact match is high and reach saturates near 1.0 for $a \leq 4$, tracking the same qualitative shape as $n = 40$), but two things differ from $n = 40$. First, reach *never really collapses*: it decays smoothly from ≈ 1.0 at $a = 1$ to ≈ 0.82 at $a = 32$, always well above the distance oracle, with none of the sharp $n = 40$ -style cliff — on this larger, sparser canvas the model leans toward *over-connecting*

rather than under-connecting once capacity is exceeded. Second, and directly following from that: cut degrades steadily and substantially, from ≈ 0.9 at small a down to ≈ 0.24 by $a = 20$ – 32 — the opposite of $n = 40$, where cut stayed at ≈ 1.0 throughout. The raw read-out logit for cut (bottom panel) makes this precise: it actually *crosses zero* and turns mildly positive past $a \approx 13$, not merely fainter as at $n = 40$ — the model is not just less confident that distant pairs are disconnected, for many of them it now positively believes they are connected. Exact match at very small a is also visibly noisier than the tight $n = 40$ result (per-seed at $a = 4$: 0.43, 0.55, 0.79, 0.00), consistent with Report VI’s own note that the $n = 64$ path_union checkpoints are a noisier, more seed-dependent read than $n = 40$ ’s tight clean model.

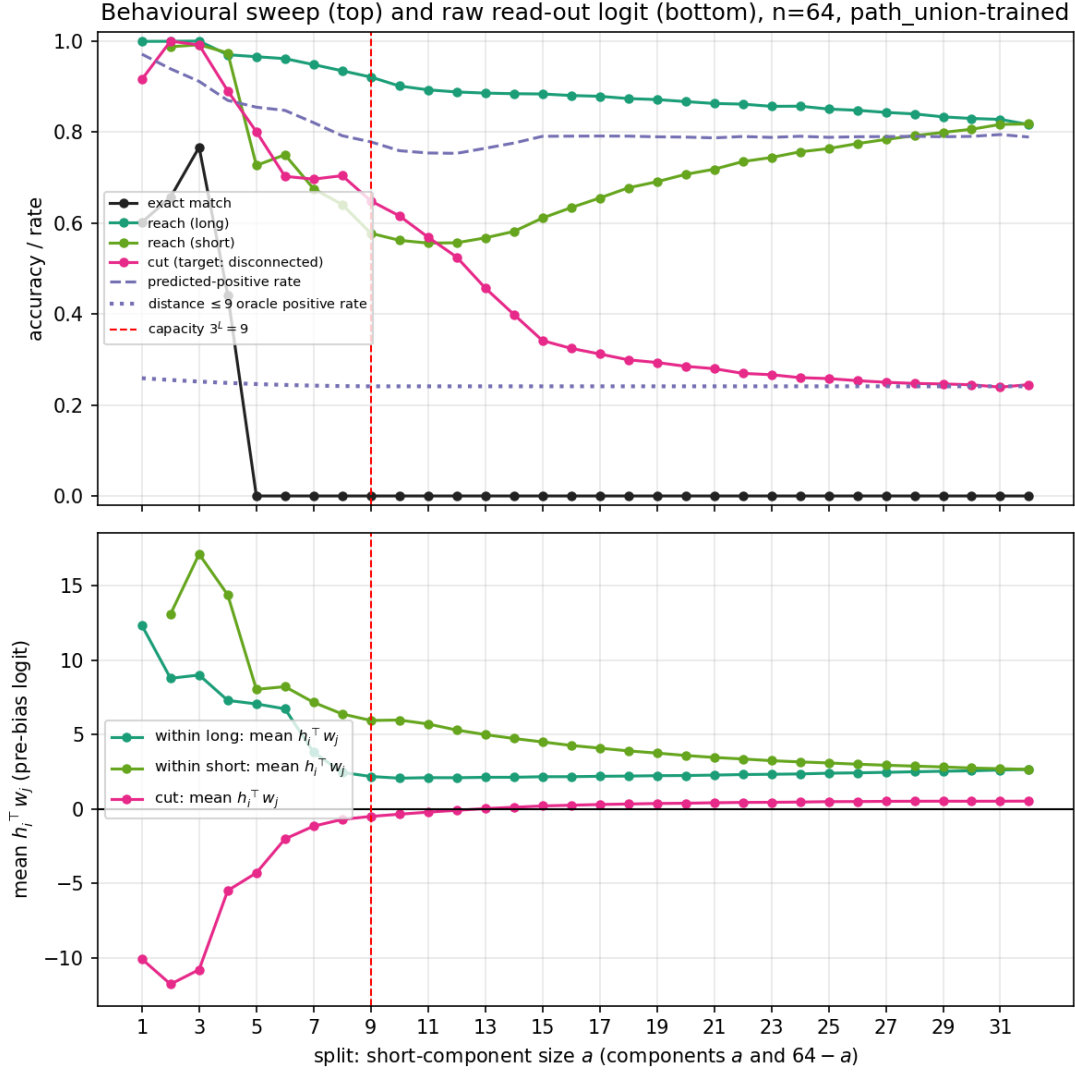


Figure 9: *Model/metrics as in Figure 8 but trained on the same disjoint-paths distribution as every other result in this report, four seeds, $n = 64$ (checkpoints from Report VI Thread A). Full split sweep $a = 1, \dots, 32$.*

The three-way falsification test flips too. Repeating §4.6 at $n = 64$ (Table 7) reproduces the ER-vs-path_union contrast cleanly. The ER-trained model keeps $\text{cut}(L_1, L_2) \geq 0.998$ at every small-component size, exactly as at $n = 40$ — the two large, unrelated components are correctly kept apart regardless of training distribution or canvas size. The path_union-trained model does *not*: at a very small third component (small = 1), it wrongly calls the two large (31- and 32-node) components connected 91% of the time ($\text{cut}(L_1, L_2) = 0.093$) — the crude

“not the small one” shortcut hypothesis 2 explicitly predicted and that $n = 40$ refuted. The error shrinks steadily as the small component grows ($0.09 \rightarrow 0.79$ from small = 1 to 10), the same direction as the two-way sweep’s cut degrading with more structure to keep separate. So hypothesis 2’s crude form, refuted at $n = 40$, is *not* refuted in general — whether the “which component” skill survives a third component depends on canvas size and training distribution, tracking the same over-connection tendency visible in Figure 9.

small	path_union-trained			ER-trained		
	reach L_1/L_2	cut(S,L)	cut(L_1,L_2)	reach L_1/L_2	cut(S,L)	cut(L_1,L_2)
1	0.926	0.999	0.093	0.396	1.000	0.998
2	0.707	1.000	0.369	0.404	1.000	0.998
4	0.756	0.999	0.322	0.417	1.000	0.998
7	0.503	0.997	0.612	0.431	1.000	1.000
8	0.427	0.997	0.717	0.438	0.999	0.999
10	0.394	0.998	0.785	0.453	1.000	1.000

Table 7: *Model/training/test as in Table 4, $n = 64$, four seeds each. Metrics:* pairwise reach inside the two large components (mean of L_1, L_2); cut between the small and either large component (mean); cut between the two large components (target disconnected — the decisive column, contrast the two blocks).

Ablation confirms layer-1 attention is a double-edged mechanism at $n = 64$: the same computation that builds the completion signal also builds the over-connection.

Figure 10 repeats §4.8 on the path_union $n = 64$ model. Zeroing layer-1 attention collapses reach just as it does at $n = 40$ (0.05–0.10 at every split) — but it also *fixes* cut, pushing it to ≥ 0.93 at every split, well above the degraded baseline. The same is true, more mildly, of zeroing layer-0 attention (cut steady at ≈ 0.88 , versus baseline’s fall to 0.24). Zeroing layer-1’s feed-forward again collapses cut specifically (down to 0.15 at $a = 32$) while leaving reach largely intact, replicating §4.8 exactly. Put together: the two-layer attention mechanism that builds the beyond-capacity reach signal is the *same* mechanism responsible for the over-connection that erodes cut on this larger canvas — removing it trades one failure for the other rather than fixing either independently.

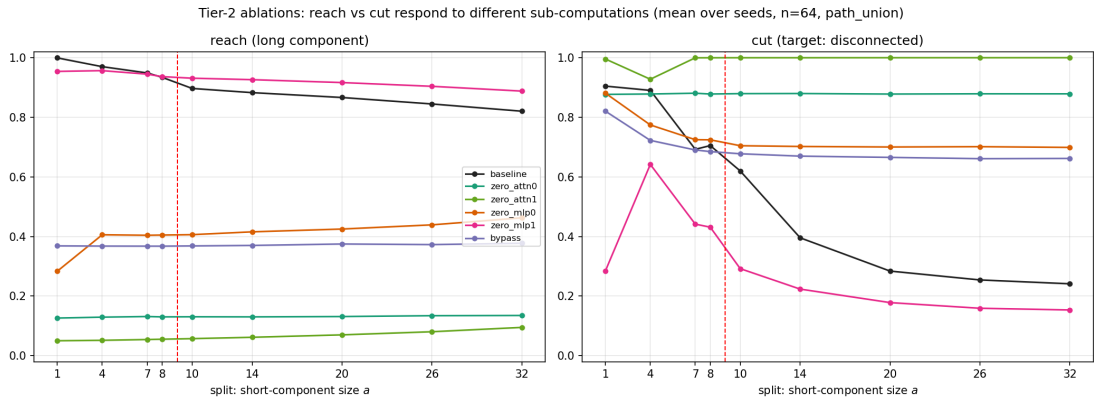


Figure 10: *Model/training/test as in Table 5, $n = 64$, path_union-trained, four seeds. Metric as in Figure 7.*

Patching, briefly. Repeating §4.9 at $n = 64$ (adjacent splits $a=7 \rightarrow a=10$, and the widest contrast $a=4 \rightarrow a=32$) reproduces the same qualitative pattern: the final-embedding patch and the random-position control land within a few points of each other in every case (0.96–1.00 vs.

0.92–0.96), the same read-out-artefact story as §4.9. Unlike at $n = 40$, the earlier (post-block-0) patch did not backfire on the mismatched pair here (1.00 for $4 \rightarrow 32$); we do not read much into that difference given the same architectural caveat already applies — it is not evidence the patch is more meaningful at $n = 64$, just that the failure mode seen once at $n = 40$ is not a robust general property of this patching design.

5 Verdict

We stated three candidate hypotheses in §1.3. The mechanistic evidence settles them as follows.

Hypothesis 3 (label/order shortcut) — refuted. Fixing node labels to a canonical order instead of relabelling every graph independently changes nothing (Table 2). The read-out matrix shows no target-specific outlier structure, and the direct read-in-to-read-out skip connection carries a signal two orders of magnitude smaller than what actually decides a prediction (Table 3) — whatever the model does, it is not reading node identity off a fixed index mapping.

Hypothesis 2 (short-component-resolved default) — correct in a narrower form than stated, and that form is itself canvas- and training-distribution-dependent. The model does not implement a generic “not in the small component therefore connected” rule: at $n = 40$, the three-way falsification test shows it correctly separates two large, unrelated components from a small one at $\text{cut} = 1.000$ throughout (Table 4), which a crude default-connect rule would get wrong. What survives at $n = 40$ is narrower and, we think, more interesting: on a graph with *exactly two* components, once one of them is small enough that the model can resolve it completely, the *other* component gets an extra, split-specific completion signal — visible as a smoothly saturating positive read-out logit (§4.3) and as attention that stays cleanly confined to the true component (§4.5) — that goes beyond plain distance-bounded reach. That signal is switched off, not merely weakened, once the graph has a third, separate component at $n = 40$: the two large components in §4.6 are each treated exactly like the long side of a balanced two-way split, not like the sole beneficiary of a completion rule. **This clean picture does not fully survive at $n = 64$** (§4.10): the same path_union-trained model, on the larger canvas, wrongly merges two large unrelated components when the third is very small ($\text{cut}(L_1, L_2) = 0.09$ at $\text{small} = 1$) — the crude form of hypothesis 2 that $n = 40$ refuted. A model trained on ER instead, at either canvas size, never shows the completion signal and never shows this failure either, keeping $\text{cut} \geq 0.998$ throughout. So “does the which-component skill generalise to a third component” is not a fixed fact about the architecture; it is entangled with how full/sparse the canvas is and what the training distribution taught, in the same direction as the over-connection tendency documented since Report III–IV for large, sparse canvases.

Hypothesis 1 (genuine, distance-graded propagation) — correct once the completion signal is not active. Beyond the two-component-with-a-small-side regime, the model’s aggregate behaviour is very hard to distinguish from plain distance-bounded matrix powering: the overall predicted-positive rate for $a \gtrsim 11$ sits almost exactly on the rate a pure distance- ≤ 9 oracle would give on the same graphs (§4.1), and the attention-leak curve (Figure 2) rises smoothly rather than switching on and off. The “genuine propagation” account is the right description of the model *outside* the specific regime where the completion shortcut fires — it is not a competing story to hypothesis 2, it is what the model falls back to when hypothesis 2’s narrow condition is not met.

Put together. The Report VI puzzle is not really about balance. It is about whether a two-component graph contains one component small enough to fully resolve. When it does,

the model gets an extra completion signal for the other component that reaches far beyond its ordinary distance-bounded capacity (the 4+36 case); when it does not — whether because the split is balanced, or because a third component is present at all — that signal does not fire, and the model’s behaviour reverts to the same distance-bounded, 3^L -capped propagation documented since Report III.

Where the signal lives. The ablations of §4.8 add a mechanistic layer to that picture rather than a competing one. Reach and cut are computed by different parts of the trunk — cut depends heavily on layer 1’s feed-forward step and barely on either attention layer’s presence *once both are intact*, while reach collapses if *either* attention layer is removed, at every split, including the completion regime. In other words, the $a \leq 7$ completion signal is not a separate sub-circuit bolted onto an otherwise-ordinary matrix-powering computation: it needs the same two-layer attention machinery that ordinary beyond-capacity reach needs, and the attention-score heatmaps (§4.7) show *why* — the same layer-1 attention that forms a narrow, capacity-bounded band at large a forms a broad, near-uniform block covering the whole long path once the other side is small enough to resolve. It is the same mechanism doing more, not two mechanisms. The activation-patching experiment (§4.9) was the one place this report tried to go further and did not cleanly succeed: because the read-out is a per-row function of a single node’s embedding, transplanting a solved graph’s embeddings into a failing one reproduces solved-like answers almost regardless of which nodes are transplanted, so that experiment could not isolate the boundary region as special. We report that as a negative methodological result, not as evidence either way.

$n = 64$ confirms the mechanism is data-shaped, not architectural, and reveals its dark side. The ER-vs-path_union contrast at $n = 64$ (§4.10) is the cleanest evidence in this report that the completion signal is something specific training distributions teach: the ER-trained model never acquires it, at any split, and tracks a plain distance-bounded oracle almost exactly throughout. The path_union-trained model does acquire it, and at $n = 64$ the ablations show it is a double-edged version of the *same* mechanism found at $n = 40$ — the identical two-layer attention computation that lets a small-side graph confirm connections far beyond capacity also, on a larger and sparser canvas, starts declaring unrelated large components connected to each other. Zeroing layer-1 attention removes both effects together (Figure 10): reach collapses, but cut is fully restored. Reach and cut were already shown to depend on different final-layer computations (cut on layer-1 feed-forward, largely independent of either attention layer’s presence); what $n = 64$ adds is that the attention mechanism which *builds* the useful completion signal is the same one that, past a point, over-generalises it into a harmful one.

6 Honest limits

This report opens the trunk further than any before it, but the evidence remains statistical and correlational, not a proof of what the network computes. The read-out, attention, and ablation analyses show quantities and sub-computations that move in lockstep with behaviour; they do not amount to a formal account of *why* training produces a completion signal specific to a two-component canvas. The activation-patching experiment (§4.9) was an attempt to localise that signal causally and came back inconclusive, for an architectural reason (the per-row read-out) rather than a negative result about the signal itself — a cleaner patching design, perhaps intervening on attention patterns directly rather than the residual stream, is left for future work. Two further follow-ups were not run in this round and are left open: a retrain with a loss that reweights within-component and cross-component pairs evenly, to separate a genuine representational effect from a training-objective artefact (the confound Report VI’s own

outlook names); and the same audit repeated with the similarity read-out, whose logit is a direct function of *both* endpoints' embeddings and may not show the same asymmetric, target-specific completion signal at all. We have not checked whether the two-component completion signal survives under the mixed training stream of earlier reports — the $n = 64$ generalisation (§4.10) checks canvas size and a from-scratch training distribution, not that stream specifically.